



Distributed Transactions

Dr. Pummy Dhiman
Assistant Professor
CUIET

The Simplicity of Local Transactions

- In a monolithic application, handling transactions is straightforward. You start a transaction, perform multiple database operations, and either commit everything or roll it all back.
- The database guarantees ACID properties: atomicity, consistency, isolation, and durability. If anything fails, the whole operation is undone as if it never happened.

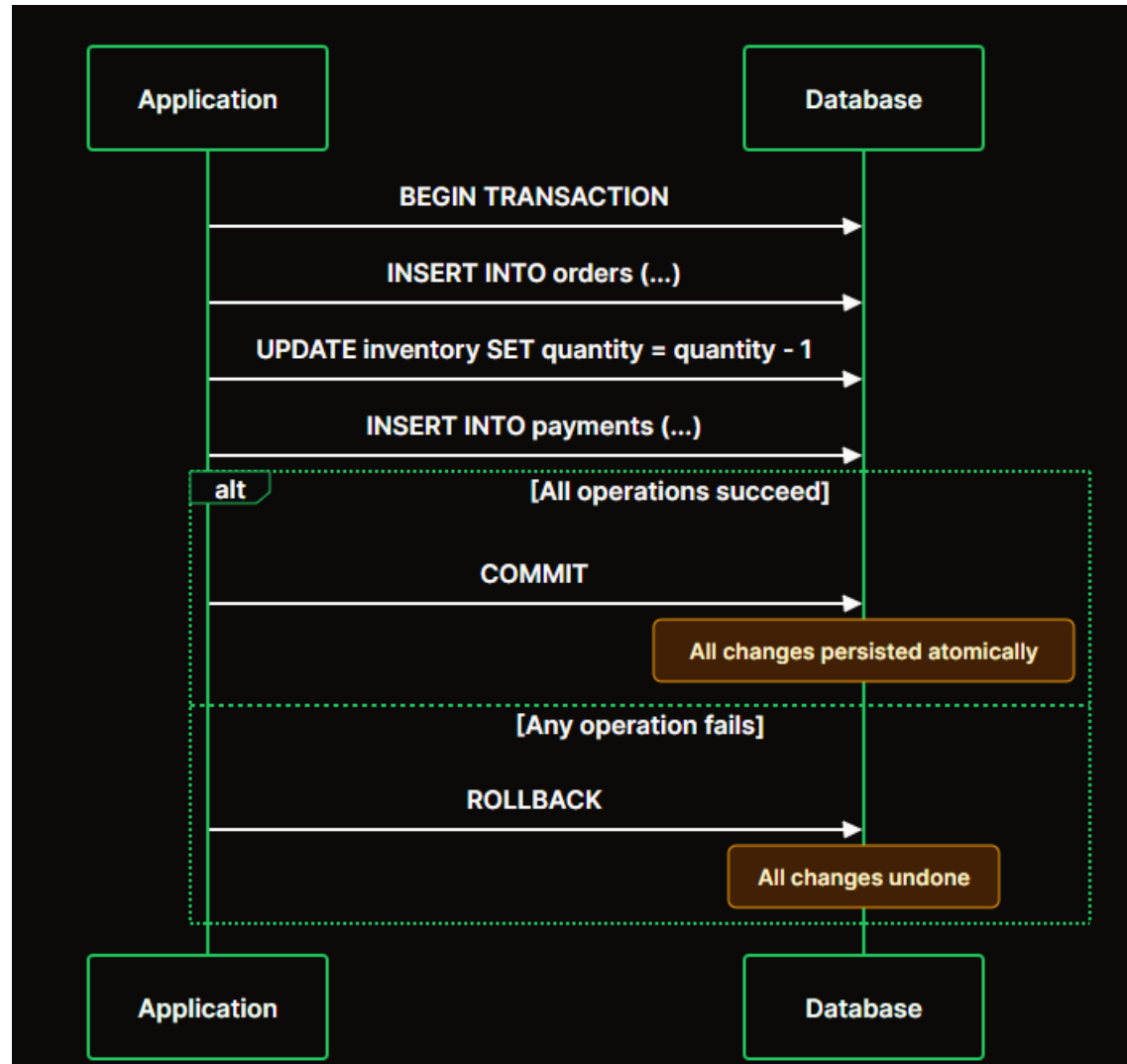
Property	Guarantee
Atomicity	All operations succeed or all fail together
Consistency	Database moves from one valid state to another
Isolation	Concurrent transactions do not interfere
Durability	Committed changes survive crashes

- This works because the database has complete control over all the data involved. It can lock rows, track changes, and coordinate the commit or rollback of the entire transaction.

The Simplicity of Local Transactions



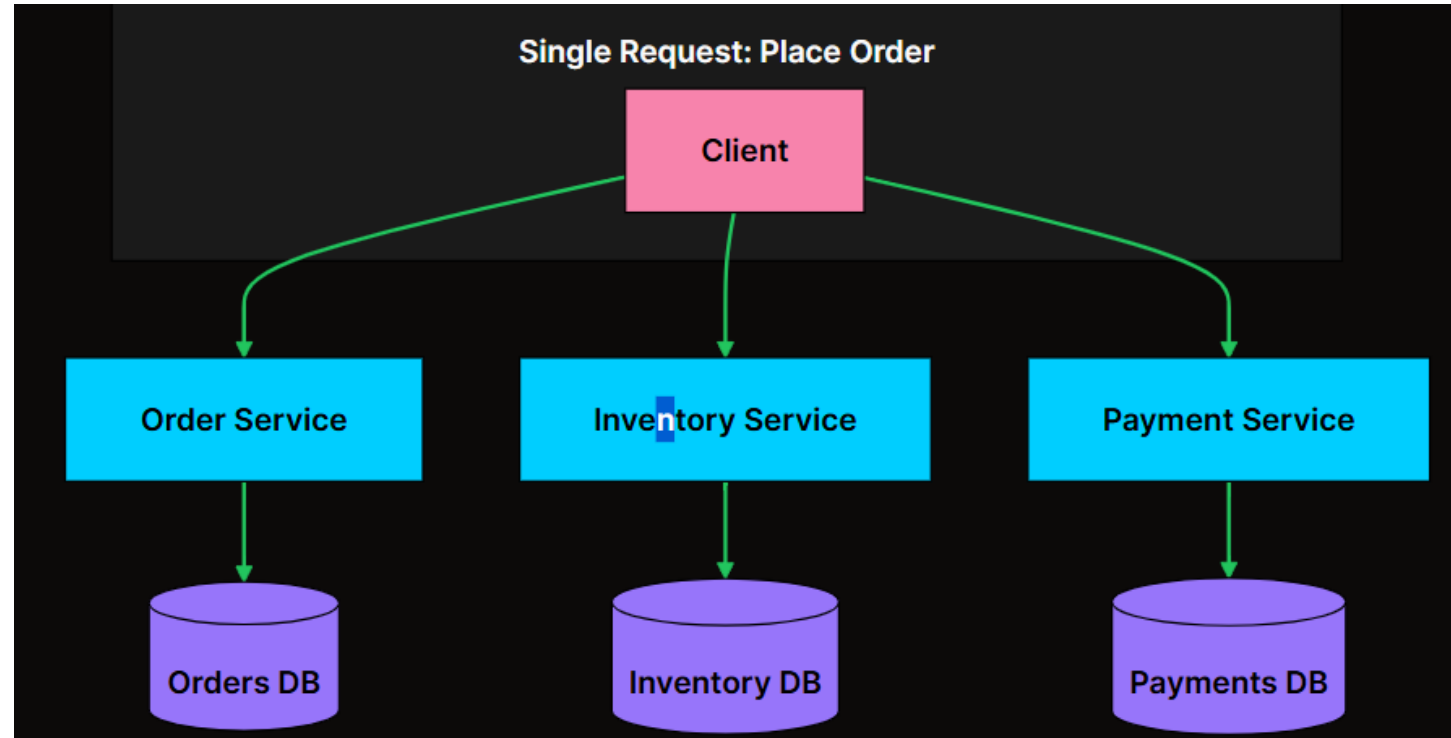
- In a single database, transactions are elegant and reliable.



The Problem with Distributed Transactions



- In a microservices system, a single business task (like placing an order) is split across multiple services, each with its own database.



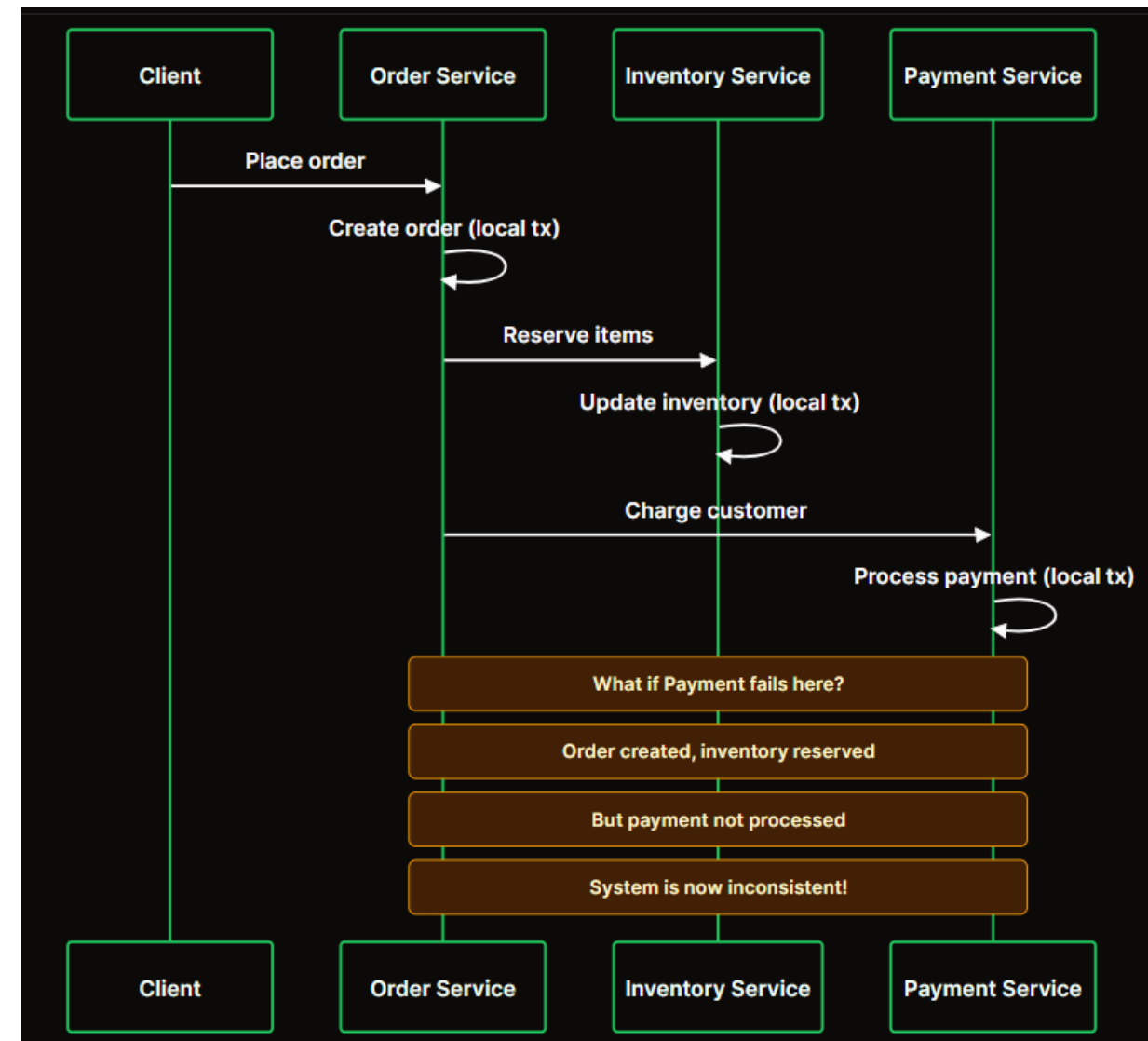
- Each service has its own database. There is no single transaction manager that controls all three databases. Each database only knows about its own local transaction.

The Problem with Distributed Transactions



The Fundamental Challenge:

- If any step fails after previous steps have committed, we have a problem. The Order Service has already committed its transaction. The Inventory Service has already committed its reservation. There is no way to roll them back.



The Problem with Distributed Transactions

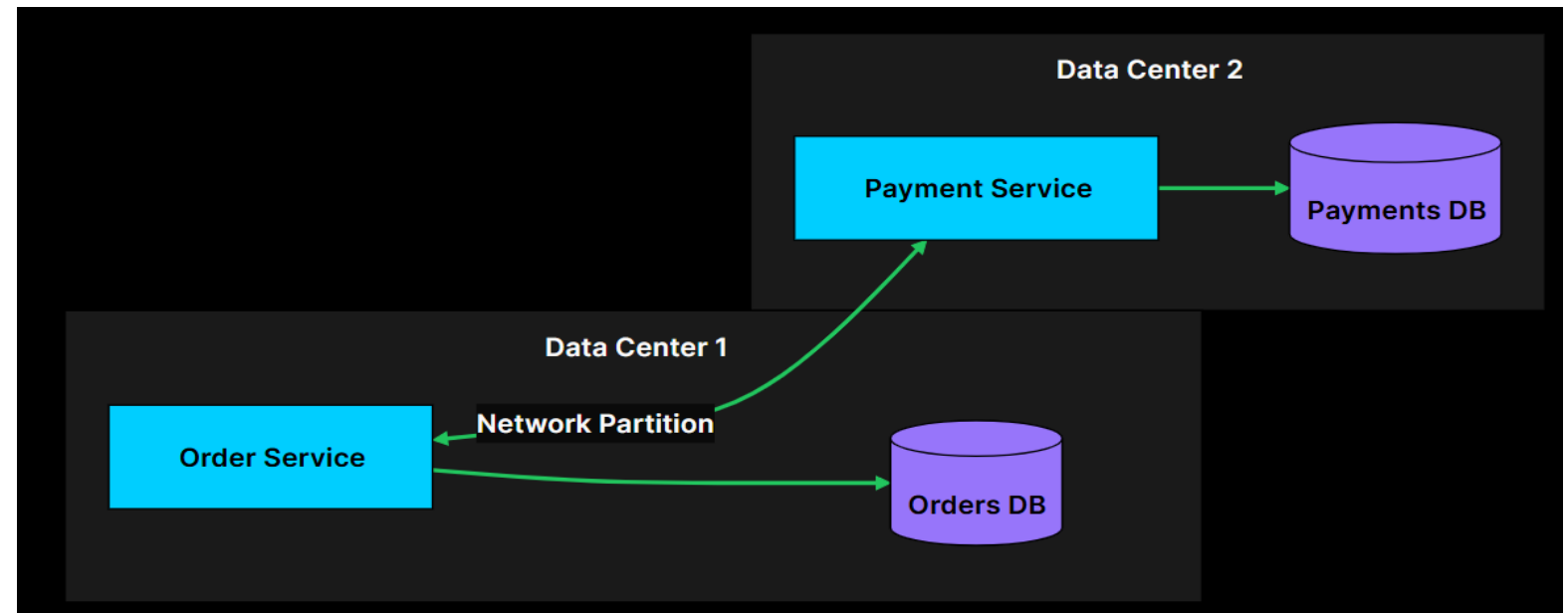


Failure Scenarios:

- Distributed systems have more failure modes than local systems. Any component can fail at any time, and the failures are often partial.

1. **Network Partitions:** When network connectivity is lost between services, Order Service cannot tell:

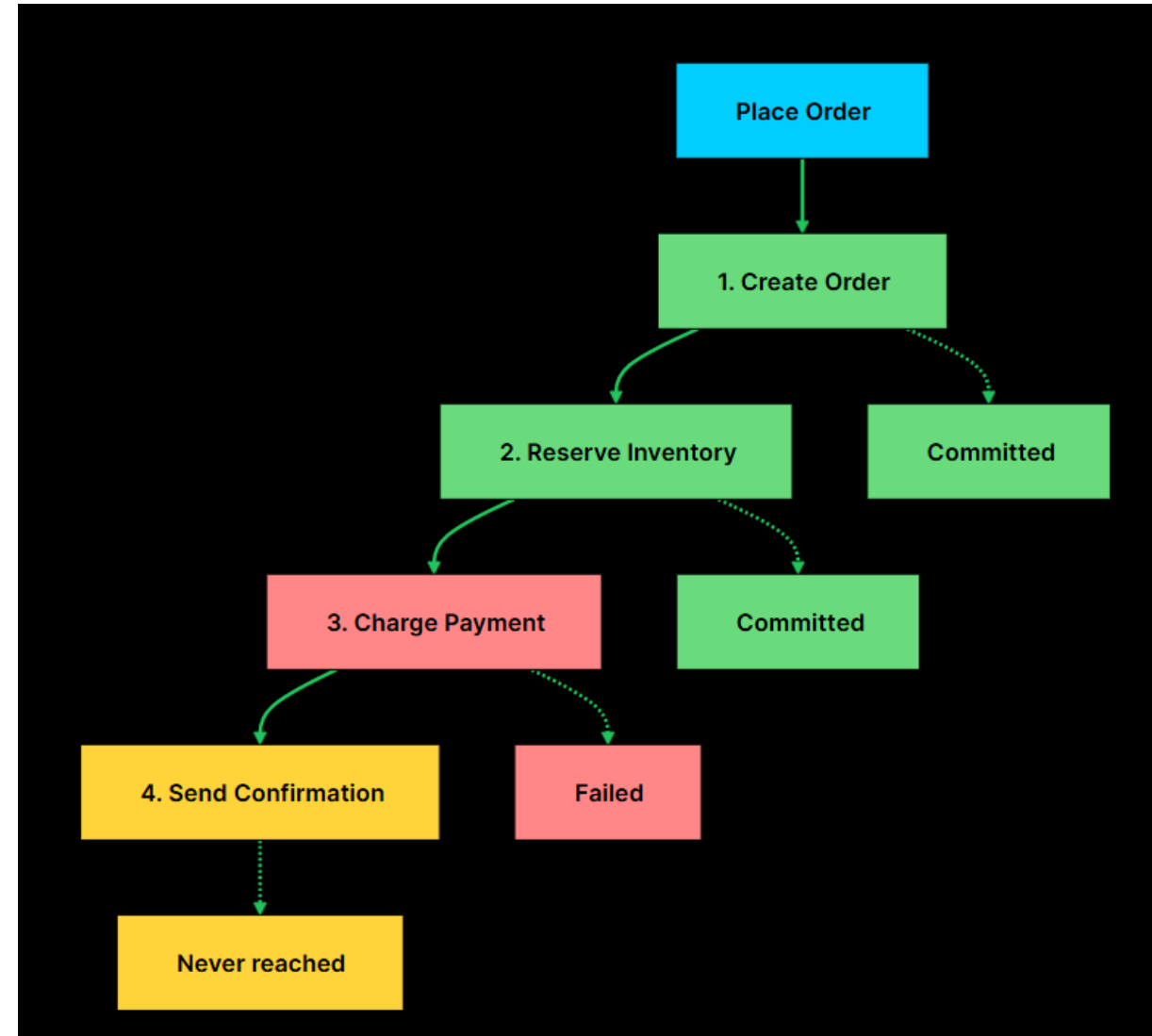
- if Payment Service is down or just unreachable.
- Did the payment succeed?
- Is it still processing?
- Should we retry?



The Problem with Distributed Transactions



2. Partial Failures: Two services succeeded and committed their changes. One failed. The system is in an inconsistent state: we have an order and reserved inventory, but no payment.



The Problem with Distributed Transactions

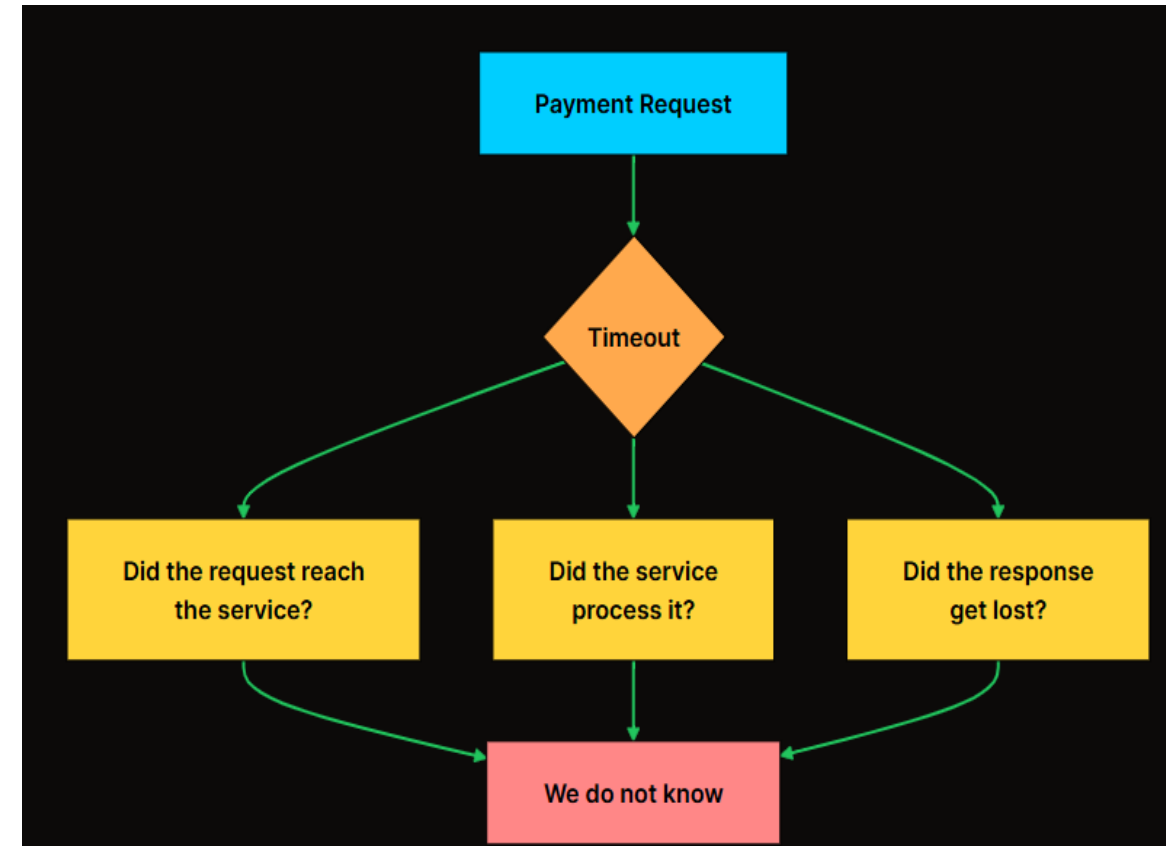


3. Timeout Ambiguity:

- One of the hardest problems in distributed systems: what does a timeout mean?

Scenario	What Happened	Our State
Request never arrived	Payment not processed	Safe to retry
Request arrived, processing failed	Payment not processed	Safe to retry
Request succeeded, response lost	Payment processed	Retry would double-charge

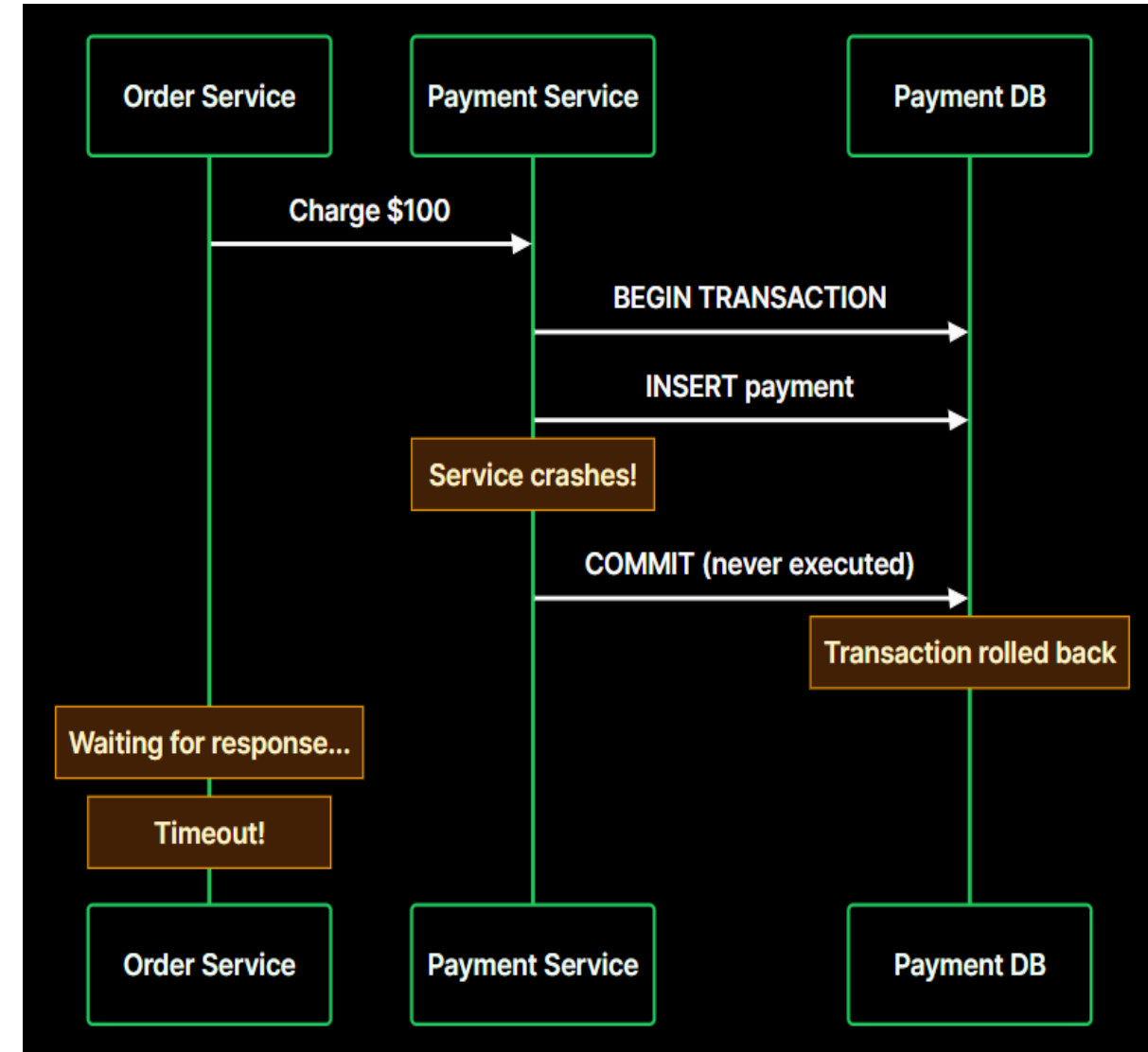
- Without additional mechanisms, we cannot distinguish these cases. This is why idempotency becomes critical in distributed systems.



The Problem with Distributed Transactions



4. Service Crashes: When a service crashes mid-operation, its local transaction rolls back automatically. But the calling service does not know what happened.



The Problem with Distributed Transactions

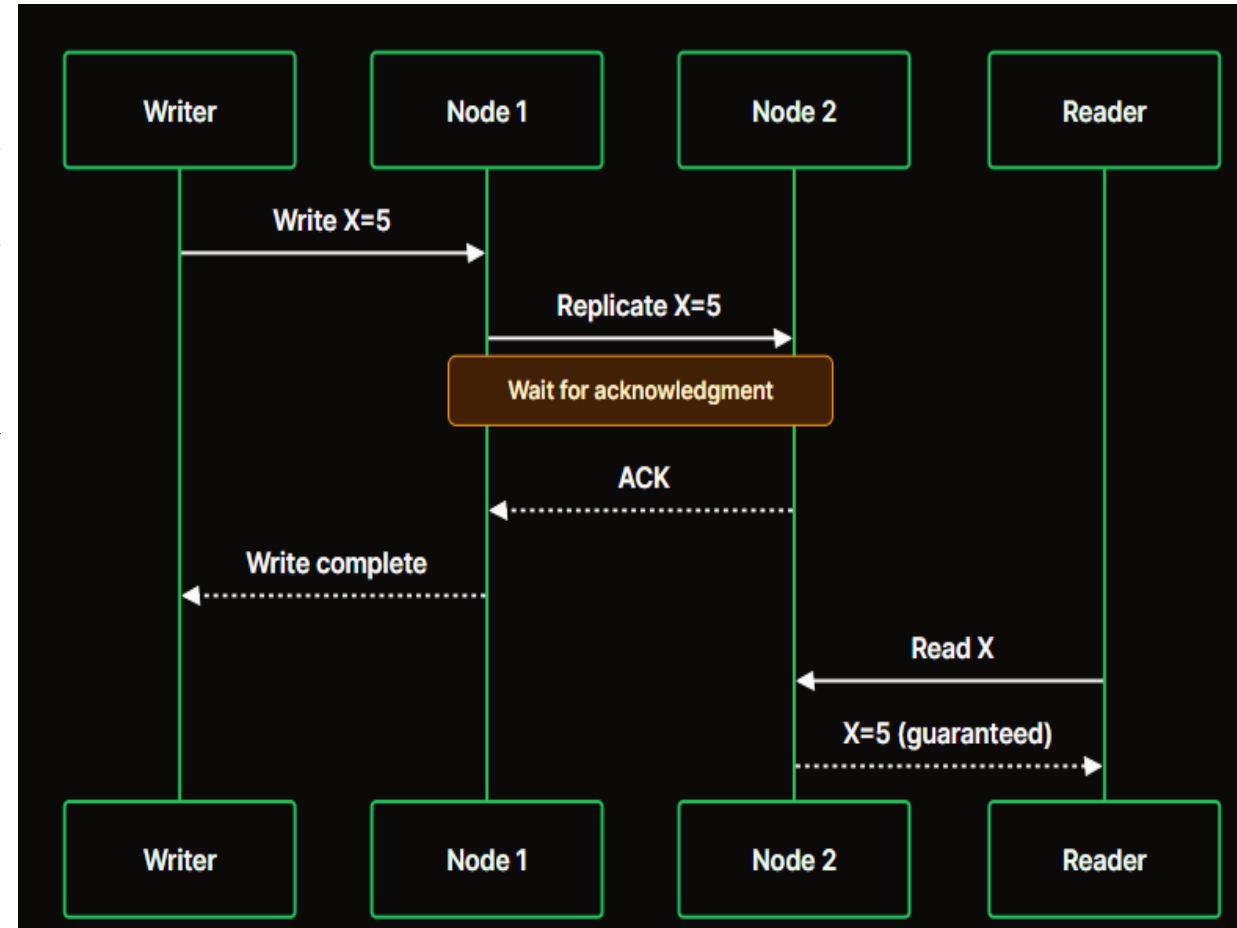


Consistency Models in Distributed Systems:

- In local transactions, we get strong consistency. In distributed systems, we often have to choose between consistency and availability.

1. Strong Consistency: All nodes see the same data at the same time. After a write completes, all readers see the new value.

Trade-off: Higher latency, reduced availability during partitions.

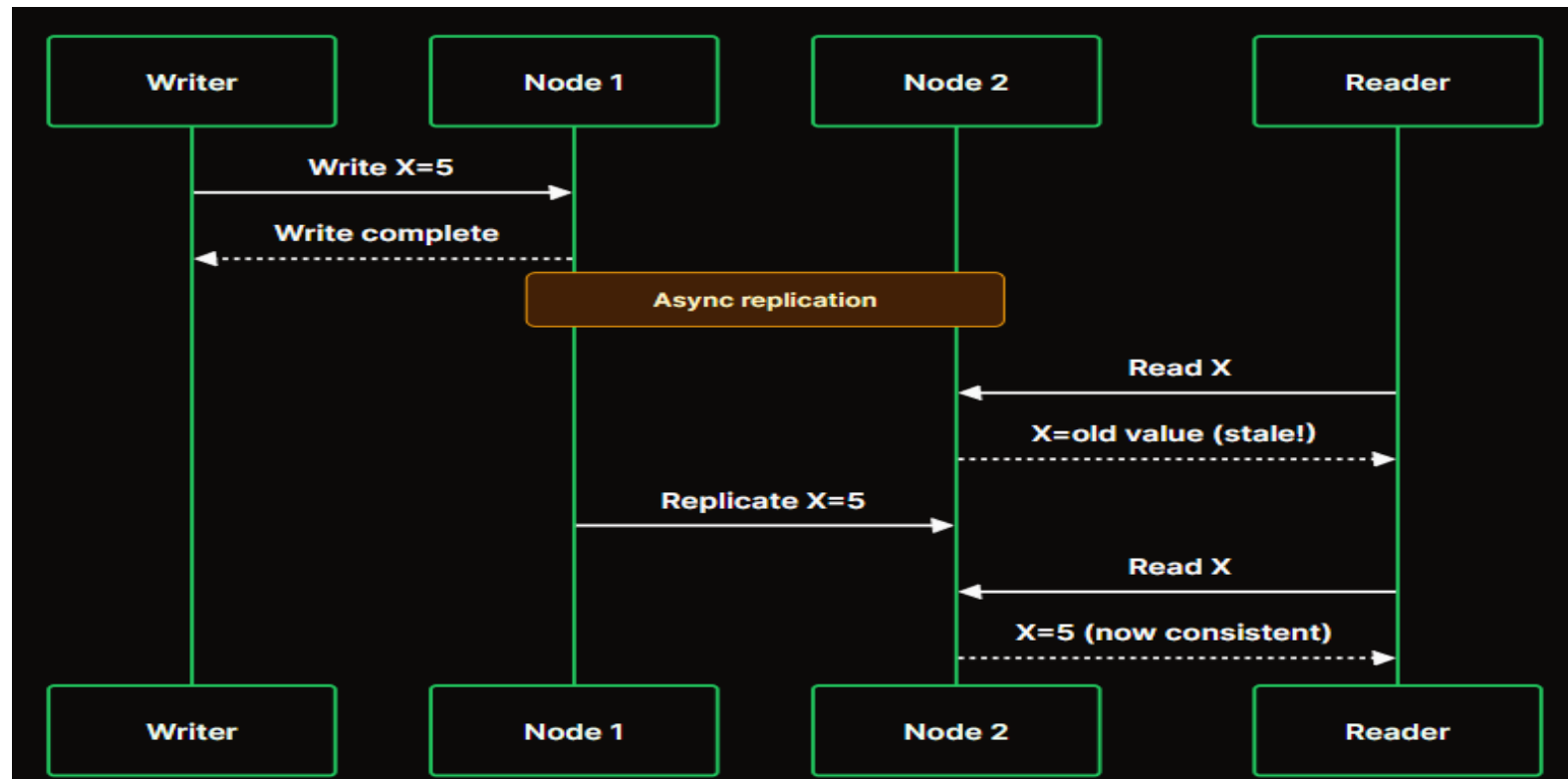


The Problem with Distributed Transactions

Consistency Models in Distributed Systems:

2. Eventual Consistency: Updates propagate to all nodes eventually, but there may be a delay. During the delay, different nodes may return different values.

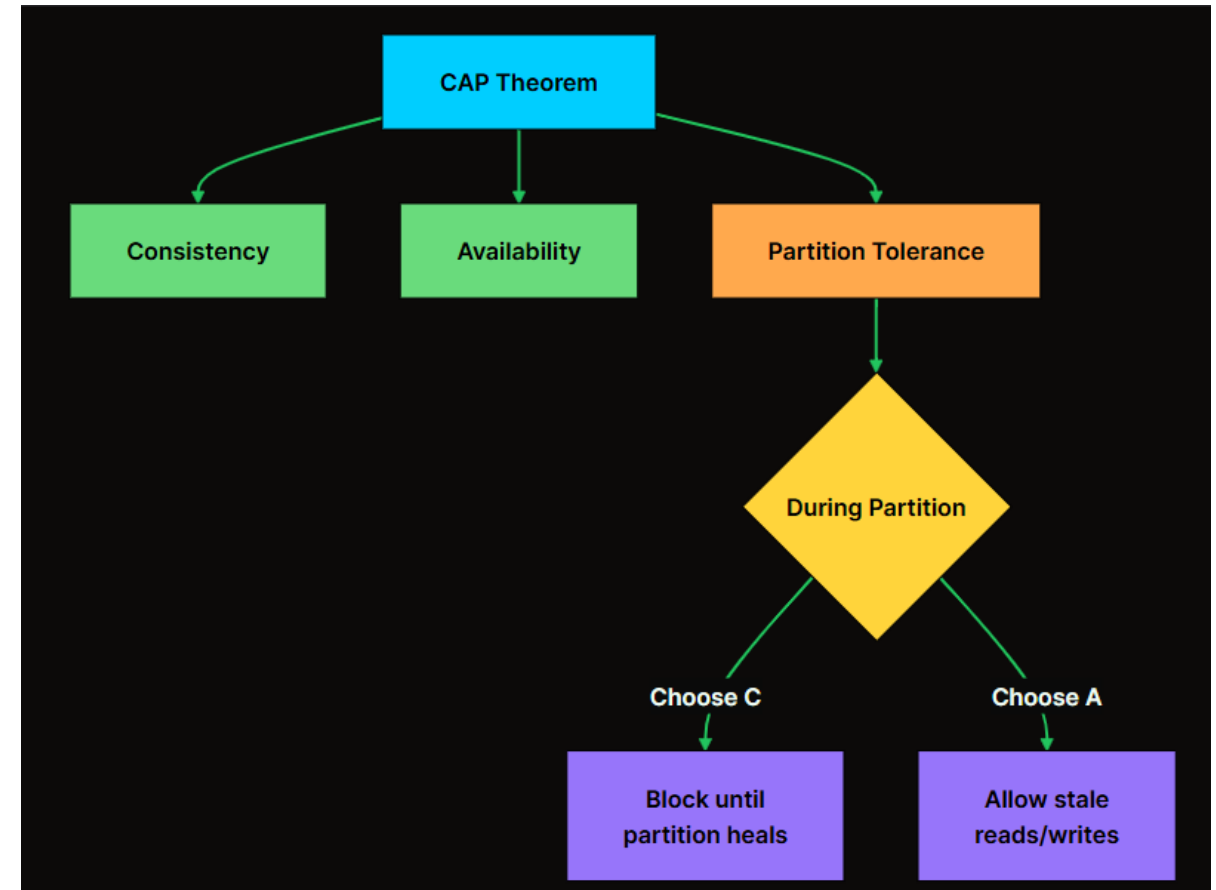
Trade-off: Lower latency, higher availability, but temporary inconsistency.



The CAP Theorem Connection



The CAP theorem states that during a network partition, a distributed system must choose between consistency and availability.



The Problem with Distributed Transactions

Several factors make distributed transactions fundamentally difficult.

1. **No Global Clock:** Each service has its own clock, and clocks drift. We cannot rely on timestamps to determine event ordering across services.
2. **No Shared Memory:** Services cannot share variables or locks. All coordination must happen through network messages, which can be lost, delayed, or duplicated.
3. **Independent Failures:** Each service can fail independently. A service might be up for some callers and down for others. It might process some requests and fail on others.

The Problem with Distributed Transactions

Given these challenges, what properties should a distributed transaction solution provide?

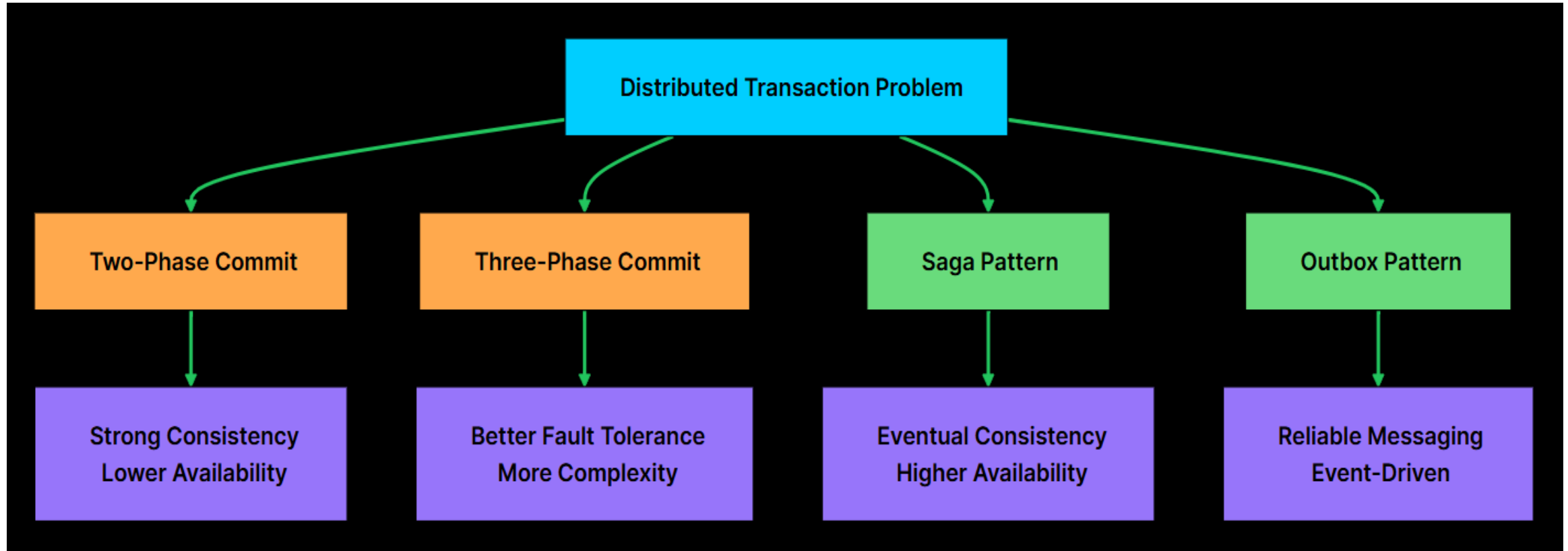
Property	Description
Atomicity	All services commit or all roll back
Consistency	System moves between valid states
Isolation	Concurrent operations do not corrupt data
Durability	Committed changes persist
Failure handling	Graceful recovery from partial failures
Performance	Acceptable latency and throughput
Availability	System remains operational during failures

- The challenge: we cannot have all properties perfectly. Every solution involves trade-offs. The patterns in the following chapters represent different points in this trade-off space.

The Problem with Distributed Transactions



Solutions:



The Problem with Distributed Transactions

Solutions:

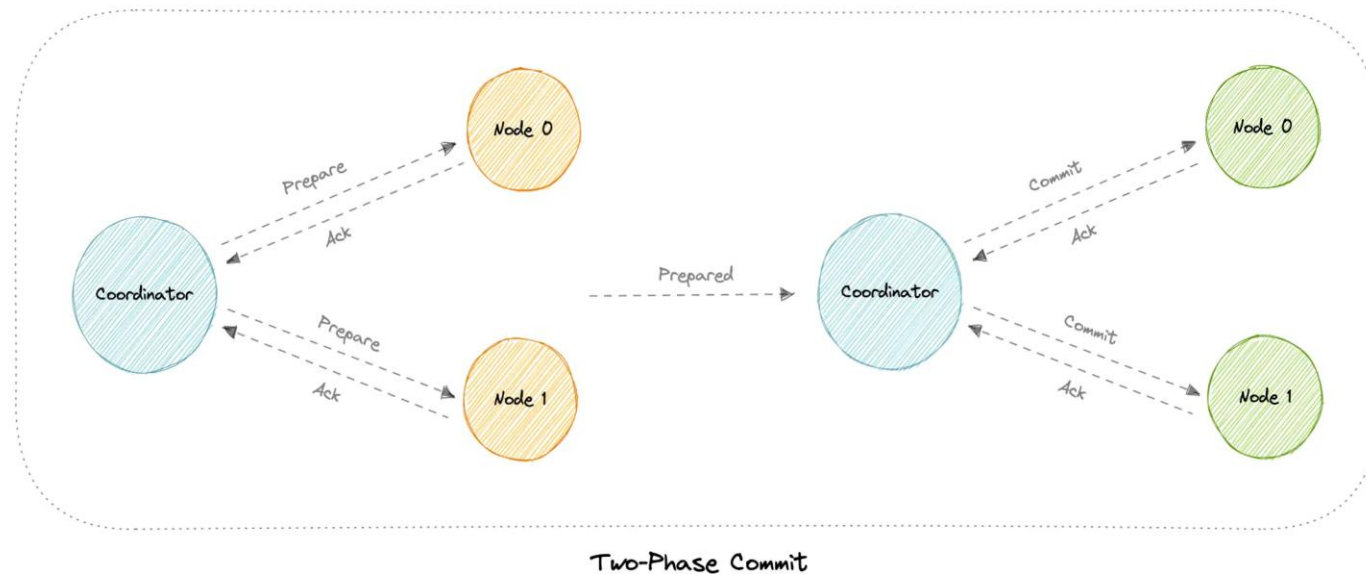
Pattern	Consistency	Availability	Complexity	Use Case
Two-Phase Commit	Strong	Lower	Medium	Tightly coupled systems
Three-Phase Commit	Strong	Better	High	Critical consistency needs
Saga	Eventual	High	Medium	Microservices
Outbox	Eventual	High	Low	Event-driven systems

Two-Phase commit

- The Two-Phase Commit (2PC) protocol is designed to ensure all nodes in a distributed system either commit or roll back a transaction. Therefore, we can achieve atomicity across multiple database nodes in the context of a distributed transaction.
- One node would act as a **coordinator**, also known as a transaction manager, to initiate the 2PC. It consists, unsurprisingly, of two phases: the preparation phase and the commit phase.

Two-Phase commit

- **Prepare phase:** The prepare phase involves the coordinator node collecting consensus from each of the participant nodes. The transaction will be aborted unless each of the nodes responds that they're prepared.
- **Commit phase:** If all participants respond to the coordinator that they are prepared, then the coordinator asks all the nodes to commit the transaction. If a failure occurs, the transaction will be rolled back.



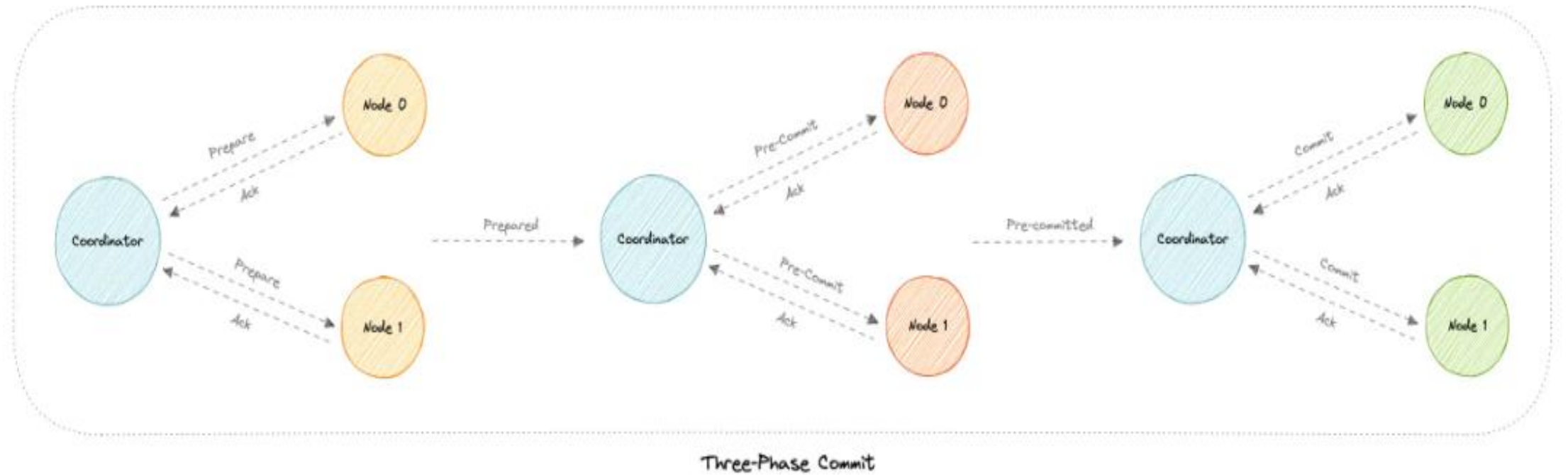
- Three-phase commit (3PC) is an extension of the two-phase commit where the commit phase is split into two phases. This helps with the **blocking problem** that occurs in the two-phase commit protocol.

➤ **Blocking Problem** : The system gets stuck waiting indefinitely if the coordinator fails at a critical moment.

Phases: Three-phase commit consists of the following phases:

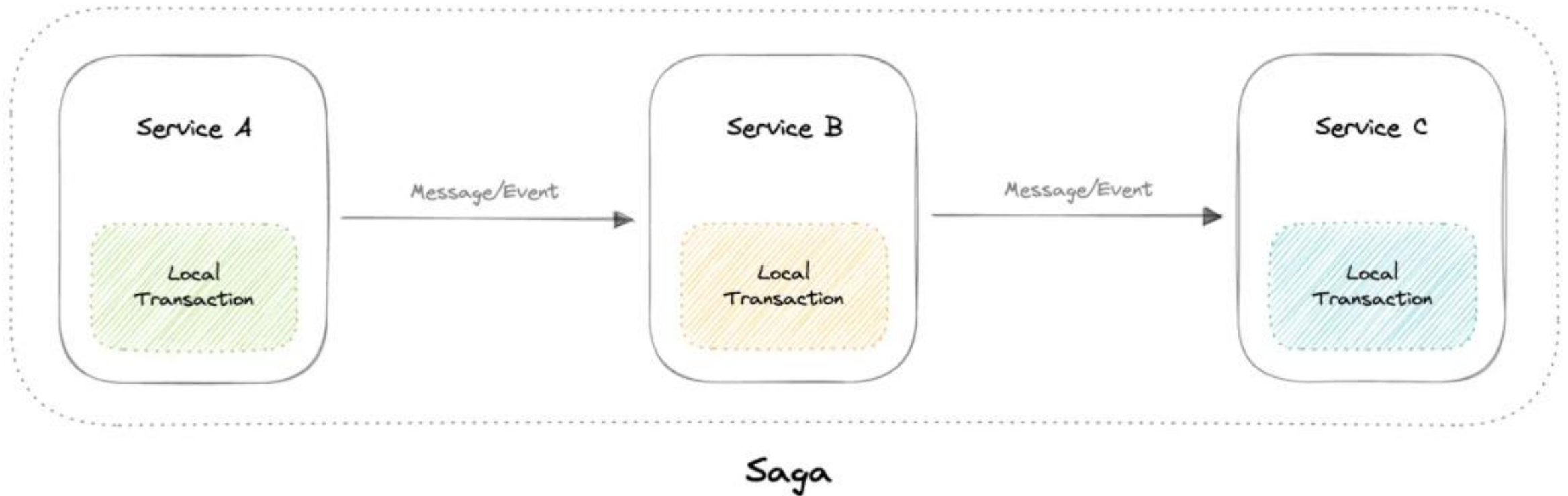
1. **Prepare phase:** This phase is the same as the two-phase commit.
2. **Pre-commit phase:** Coordinator issues the pre-commit message and all the participating nodes must acknowledge it. If a participant fails to receive this message in time, then the transaction is aborted.
3. **Commit phase:** This step is also similar to the two-phase commit protocol.

Three-Phase commit



- The Saga Pattern is an alternative approach to managing distributed transactions, especially useful in microservices architecture where long-lived transactions are involved.
- Instead of ensuring atomicity through a single, all-or-nothing transaction, this pattern decomposes a transaction into a series of smaller, independent sub-transactions, also called local transactions.
- Each sub-transaction is managed by a separate service, and together, they form a saga. Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga.
- If a local transaction fails, the saga executes a series of compensating transactions to undo the changes that were made by the preceding local transactions.

Saga Pattern



- In sagas, we have three different types of transactions:
 - 1. Compensable Transactions:** transactions that can potentially be reversed by processing another transaction with the opposite effect.
 - 2. Pivot Transaction:** the go/no-go point in a saga. If the pivot transaction commits, the saga runs until completion. A pivot transaction can be a transaction that is neither compensable nor retryable. Also, it can be the last compensable transaction or the first retryable transaction in the saga.
 - 3. Retryable Transactions:** transactions that follow the pivot transaction and are guaranteed to succeed.

- There are two common saga implementation approaches: **choreography and orchestration**.

1. Choreography: In a choreography-based approach, each local transaction publishes events that trigger local transactions in other services. Therefore, no central coordinator is telling the saga participants what to do.

Example Workflow:

1. Order Placement Service creates an order and publishes an event
2. Payment Service receives the event, processes the payment, and publishes a confirmation event
3. Inventory Service receives the event, updates the inventory, and publishes a confirmation event
4. Delivery Service receives the event and schedules the delivery

If any step fails, each service involved must execute a compensating transaction to revert its changes.

2. Orchestration : In an orchestration-based saga, a central orchestrator (or coordinator) manages the entire transaction. The orchestrator sends commands to each service to perform their local transactions. It also handles any failures by sending commands to execute compensating transactions as necessary.

Example Workflow:

1. Order Placement Service can be considered as an orchestrator. Once it creates an order, it commands the Payment Service to process the payment
2. After the payment is processed, the orchestrator instructs the Inventory Service to update the inventory
3. Finally, the orchestrator tells the Delivery Service to schedule the delivery

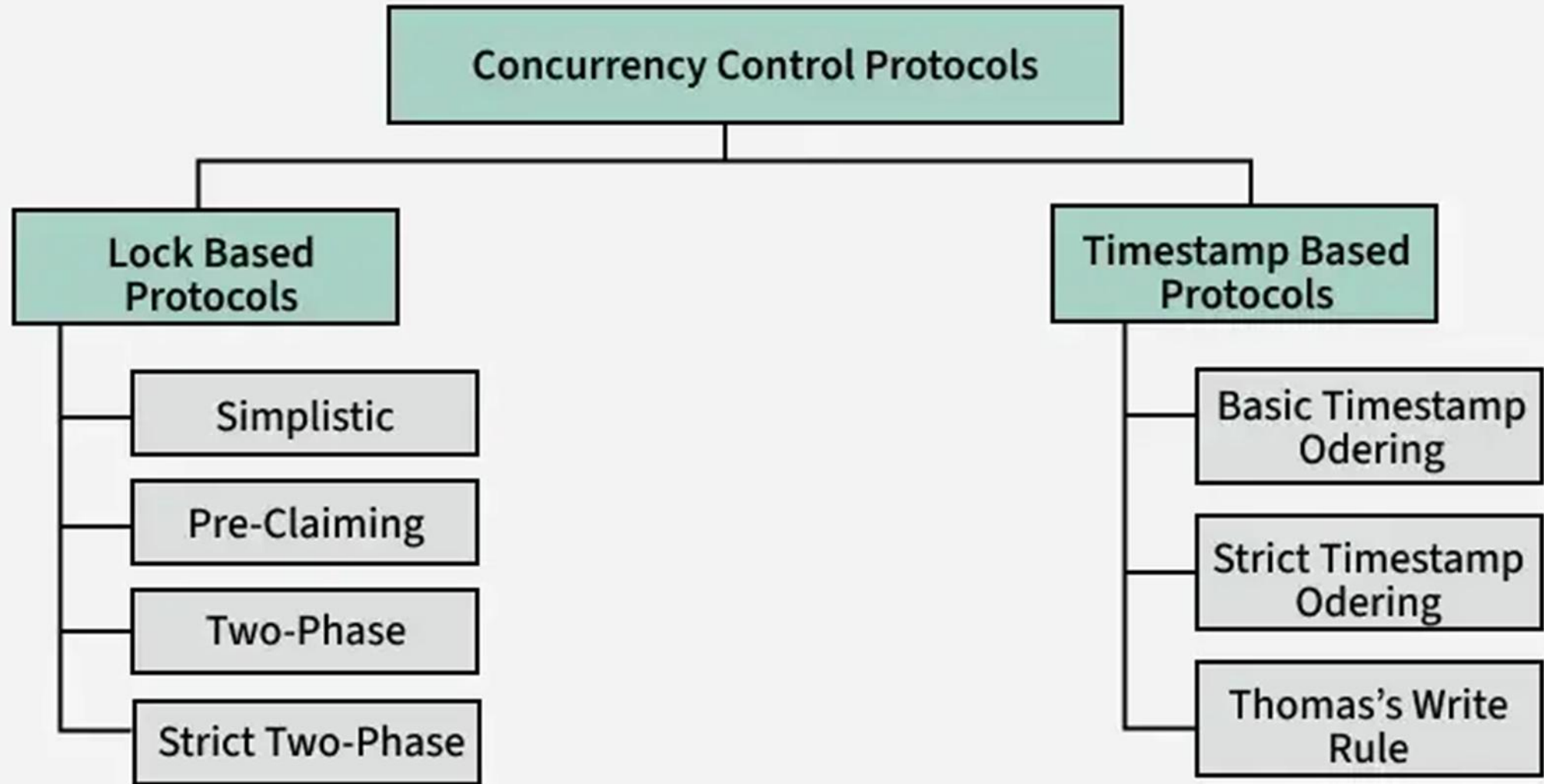
If any sub-transaction fails, the orchestrator sends commands to undo the preceding steps.

Need of Concurrency Control

Scenario	Without Concurrency Control	With Concurrency Control
Initial Balance	\$1000	\$1000
Transaction A (User 1)	Withdraws \$200: Reads balance as \$1000, calculates new balance \$800.	Withdraws \$200: Reads balance as \$1000, calculates new balance \$800.
Transaction B (User 2)	Withdraws \$300: Reads balance as \$1000 (before A updates), calculates \$ 700	Waits for Transaction A to finish. Once A commits, reads updated balance \$800, calculates new balance \$500.
Final Updates	A writes \$800. Then, B reads overwrites with \$700 (lost update issue).	A writes \$800. Then, B reads updated balance and writes \$500 (consistent result).
Final Balance	\$700 (incorrect, inconsistent).	\$500 (correct, consistent).

- The concurrency control protocols ensure the atomicity, consistency, isolation, durability and serializability of the concurrent execution of the database transactions. Therefore, these protocols are categorized as:
 - **Lock Based Concurrency Control Protocol**
 - **Time Stamp Concurrency Control Protocol**
 - **Validation Based Concurrency Control Protocol**

Concurrency Control Protocols



- Two-phase locking (2PL) is one of the most widely used concurrency control protocols, particularly in centralized, single-node relational databases.
- It requires transactions to acquire locks before accessing data. Lock-based protocols, in general, obtain a lock before they operate on an item. 2PL is a type of lock-based protocol that employs two types of locks:
 - **Read lock (shared lock):** Allows multiple transactions to read an item simultaneously.
 - **Write lock (exclusive lock):** Prevents other transactions from acquiring any lock (read or write) on the same item.

This locking behavior aligns with the notion of conflicts introduced in the previous section.

Two-phase locking (2PL)

2PL consists of two phases:

- **Growing phase:** The transaction acquires locks but does not release any.
- **Shrinking phase:** Once a transaction releases a lock, it cannot acquire any new locks.

The moment when a transaction has acquired all its required locks is known as the lock point. The schedule produced by 2PL is equivalent to a serial schedule where transactions are ordered by their lock points.

Timestamp based protocols

- In a timestamp-based protocol, each transaction T is assigned a unique, monotonically increasing timestamp, $TS(T)$, upon entering the system.
- This timestamp can be derived from a logical counter that increments with each new transaction. The protocol ensures the resulting schedule is equivalent to a serial execution ordered by transaction timestamps.

Mechanism

Each data item maintains two timestamps:

- **Read timestamp (RTS):** The largest timestamp of any transaction that has successfully read the item.
- **Write timestamp (WTS):** The largest timestamp of any transaction that has successfully written the item.

The execution rules are as follows:

- **Read operation:** A transaction T can read an item X only if $WTS(X) \leq TS(T)$ (i.e., the item was last written by an earlier or the same transaction). If the read is successful, $RTS(X)$ is updated to $\max(RTS(X), TS(T))$.
 - **$WTS(X) \leq TS(T)$**
 - **$WTS(X)$ = Write Timestamp of data item X (the last time X was written).**
 - **$TS(T)$ = Timestamp of transaction T .**
 - **Meaning:**
Transaction T can read item X only if X was last written by an older transaction or the same transaction.
- $WTS(X) = 5$
- $TS(T) = 10$
- Since $5 \leq 10$, transaction T can read X .

- **Write operation:** A transaction T can write an item X only if $RTS(X) \leq TS(T)$ and $WTS(X) \leq TS(T)$ (i.e., no transaction with a later timestamp has read or written the item). If the write is successful, $WTS(X)$ is updated to $TS(T)$.

- **$RTS(X) \leq TS(T)$**

- **$RTS(X)$** = Read Timestamp of X (largest timestamp of a transaction that has read X).

- **$TS(T)$** = Timestamp of transaction T .

- **Meaning:**

Transaction T can write item X only if **no younger transaction has already read X** .

$RTS(X) = 6,$

$TS(T) = 10$

Since, $6 \leq 10$, so write allowed.

These rules ensure that transactions execute in a manner consistent with their timestamp order, thereby guaranteeing serializability.

- Transactions ordered globally so that older transactions, transactions with smaller timestamps, get priority in the event of conflict.
- Conflict is resolved by rolling back and restarting transaction.
- No locks so no deadlock.
- Read/write proceeds only if last update on that data item was carried out by an older transaction.
- Otherwise, transaction requesting read/write is restarted and given a new timestamp.

- Pure timestamp-based protocols are rarely used in isolation due to their overhead, particularly the need to maintain and update timestamps even for read operations.
- However, the concept of assigning timestamps to data items and transactions is fundamental to multi-version concurrency control (MVCC), a widely used approach.

Advantages of Timestamp Ordering

- **High Concurrency:** Allows for a high degree of concurrency as transactions with non-conflicting operations can proceed simultaneously.
- **Determination of Serializability:** The protocol guarantees that the execution of transactions is equivalent to some serial order, ensuring serializability.
- **No Deadlocks:** Timestamp Ordering eliminates the possibility of deadlocks because transactions are strictly ordered based on timestamps.

- Security in distributed systems can roughly be divided into two parts.
- One part concerns the communication between users or processes, possibly residing on different machines. The principal mechanism for ensuring secure communication is that of a secure channel.
- The other part concerns authorization, which deals with ensuring that a process gets only those access rights to the resources in a distributed system it is entitled to.
- Secure channels and access control require mechanisms to distribute cryptographic keys, but also mechanisms to add and remove users from a system.

- Distributed systems consist of multiple interconnected machines that communicate over a network. Ensuring data security in such systems is challenging because:
- Remote machines may not follow the same security policies.
- The communication network is untrusted and vulnerable to attacks such as interception, modification, replay, or deletion of messages.
- Thus, protecting data requires strong mechanisms like cryptography, authentication, and secure communication protocols.

1. Confidentiality

- Ensures that data is accessible only to authorized users
- Prevents unauthorized disclosure of information

Techniques:

- Encryption (AES, RSA)
- Secure communication protocols

Key Security Requirements

Integrity

- Ensures that data is not altered or tampered with
- Maintains accuracy and consistency

Techniques:

- Hash functions (SHA)
- Digital signatures

Availability

- Ensures that data and services are accessible when needed
- Protects against failures and attacks (like DoS)

Authentication

- Verifies identity of users and systems
- Methods:
 - Passwords
 - Biometrics
 - Multi-factor authentication (MFA)

Authorization

- Determines what actions a user is allowed to perform
- Example:
 - Read-only vs full access

Common Security Threats:

- **Eavesdropping** → Unauthorized interception of data
- **Data tampering** → Modification of data
- **Replay attacks** → Reusing old messages
- **Denial of Service (DoS)** → Overloading system resources
- **Man-in-the-middle attack** → Intercepting communication between nodes

1. Encryption

- Protects data during transmission and storage
- Types:
 - Symmetric encryption (fast)
 - Asymmetric encryption (secure key exchange)

2. Secure Communication

- Use protocols like:
 - SSL/TLS
 - HTTPS

3. Access Control

- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)

4. Firewalls & Intrusion Detection

- Firewalls filter network traffic
- IDS/IPS detect suspicious activities

5. Replication & Backup

- Data is replicated across nodes
- Helps in:
 - Fault tolerance
 - Data recovery

Key Security Challenges

- Untrusted Nodes: Systems must interact with machines that may be compromised or malicious.
- Insecure Communication Network: Attackers can:
 - Eavesdrop on messages
 - Modify or forge data
 - Replay old messages
- Lack of Central Control: No single authority controls all machines in a distributed environment.

- Authentication ensures that communicating parties are genuine.
- Common Methods:
 - Password-based Authentication
 - User proves identity using a secret password.
- Requires encryption during transmission.
- Public Key Authentication:
 - Uses public-private key cryptography.
 - More secure for large-scale distributed systems.
- Authentication helps ensure:
 - Data comes from a trusted source
 - Communication is not forged

- To verify identities in distributed systems:
- Certificates are used.
- Issued by trusted entities called Certificate Authorities (CAs).

How it works:

1. A CA verifies an entity.
2. It issues a digital certificate containing:
 - Public key
 - Identity information
 - Digital signature
3. Others can verify authenticity using the CA's public key.
4. Advantage: No need for direct trust between communicating parties.

Types of Cryptography:

- **Asymmetric (Public Key)**
 - Used for authentication and key exchange
 - Slower but secure
- **Symmetric Key**
 - Used for encrypting actual data
 - Faster and efficient

SSL/TLS provides secure communication over networks.

- **Features:**

- Encryption of data
- Authentication of server (and optionally client)
- Data integrity protection

- **Working:**

- Client and server negotiate security parameters
- Server sends certificate
- Secure key exchange (often using Diffie-Hellman)
- Encrypted communication begins

SSL/TLS provides secure communication over networks.

- **Features:**

- Encryption of data
- Authentication of server (and optionally client)
- Data integrity protection

- **Working:**

- Client and server negotiate security parameters
- Server sends certificate
- Secure key exchange (often using Diffie-Hellman)
- Encrypted communication begins